CIC-307 | Computer Networks | Mini Project

Maharaja Agrasen Institute of Technology
Department of Computer Science and Engineering

NetArchitect

A network planning and Cisco IOS configuration engine that turns a campus, metro, or WAN scenario into explained design decisions, a VLSM address plan, and Packet Tracer configs.

Submitted by

Aayush Kumar

Roll number 08514802724
B.Tech Computer Science and Engineering

Source code: https://github.com/Aayush3238/CN_Assignment
November 2026

All Cisco IOS in this report is generated by the project itself. Lab credentials are intentionally non-production.

1. Abstract

NetArchitect is a Python planning tool for CIC-307. A scenario, either one of three built-in presets or a custom JSON file, is passed through media selection, bitwise VLSM, topology sizing, and routing selection. Each choice is returned as a decision that cites the input numbers and the standard that justifies it. The same result is rendered as Cisco IOS for Packet Tracer 8.x and shown on a local web dashboard. The subnet arithmetic does not use Python's ipaddress module; the tests cross-check the allocator against that module.

The campus preset is a 1,600-endpoint, three-building LAN and is planned as three-tier with single-area OSPF. The metro preset connects a headquarters to two remote sites over 15 km and 35 km and is planned as a collapsed core with multi-area OSPF. The WAN preset covers four cities and a cloud VPC and is planned as three-tier with multi-area OSPF plus eBGP. The suite contains 128 tests, and ai_audit_log.md records four real defects found in AI-generated code, including a cloud prefix leaked into OSPF and a duplicate OSPF router ID on an HSRP pair.

2. Objectives

- Accept three realistic presets and a custom scenario, and reject distances, counts, and prefixes that are missing, non-numeric, duplicate, or out of range.
- Explain every media type, transceiver, routing protocol, topology tier, and switching choice in natural language tied to the scenario numbers.
- Allocate subnets with bitwise VLSM, with no overlap, correct wildcard masks, and explicit failure on exhaustion.
- Emit copy-oriented Cisco IOS for switches, routers, SVIs, trunks, HSRP, OSPF, BGP, ACLs, QoS, and IPsec.
- Expose the same analysis through a CLI and a browser dashboard without recalculating any value twice.
- Lock the rules in with automated tests, and record real generation mistakes with a packet-flow explanation.

3. Tools

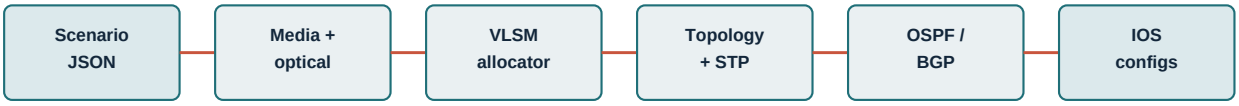
Piece	Role
Python 3.10+	Engine, CLI, dashboard, and tests. The core has no web imports.
pytest	Unit and integration tests for media, VLSM, routing, topology, switching, configs, CLI, and API.
Jinja2	One IOS template rendered per device, with strict undefined variables.
FastAPI and Jinja2	Dashboard at http://127.0.0.1:8000. No frontend build step.
Rich	Coloured CLI tables for decisions, the address plan, and errors.
Cisco Packet Tracer 8.x	Lab used to paste exports/campus.txt, metro.txt, and wan.txt.
reportlab	Builds this PDF from the live engine at report time.

Table 1. Tools and their role.

4. Architecture

A scenario is a JSON object with a name, endpoint count, building count, links, subnet requests, a root prefix, sites, and flags for a single autonomous system, a cloud boundary, and resiliency. core/spec.py parses that object and rejects a blank name, a boolean where an integer is required, a zero or non-numeric distance, a duplicate VLAN, a subnet claimed by two sites, and a pricing block that does not include all three switch prices.

design() runs media selection on every link, allocates the VLSM plan, sizes the topology, and then plans routing from those results. analyze() also builds a second architecture so the dashboard can show the selected design beside an alternate. The CLI and the web API both call analyze(); they do not recalculate masks or optical loss themselves.



one Analysis object feeds the export, the CLI, and the dashboard

Figure 1. Analysis pipeline shared by every interface.

Module	Responsibility	Output
core/spec.py	Parse and validate scenarios; load presets.	Scenario
core/ipv4.py	Bitwise address arithmetic without ipaddress.	int helpers
core/media.py	Distance rules, transceiver choice, optical budget.	MediaChoice
core/vlsm.py	Descending sort, boundary alignment, overlap scan.	VLSMPlan
core/topology.py	Tier rule, switch counts, oversubscription.	TopologyPlan
core/routing.py	Area mapping, summaries, eBGP, cloud isolation.	RoutingPlan
core/switching.py	RSTP/MSTP, HSRP/VRRP, hardening list.	SwitchingPlan
core/design.py	Orchestration, alternate plan, violations.	Analysis
configs/configs.py	Device model and Cisco IOS rendering.	dict[str, str]

Table 2. Module responsibilities.

A Decision carries a category, the choice, a reason, the standard, a severity, and an evidence dictionary. Calculations such as subnet addresses and optical budgets live on the plan object next to the decision, so the interface can show the numbers the engine actually computed. Each run writes three artifacts from that one object.

Artifact	Content	Consumer
.txt	Combined Cisco IOS, one banner-delimited block per device.	Packet Tracer operator
-plan.json	Scenario, media, VLSM, topology, routing, switching, decisions, violations, configs.	Automated checker
-rationale.md	Scenario, selected design, media table, decision log, address plan, alternate design.	Report reader

Table 3. Artifacts written by one run.

5. Media selection

Distance picks the medium. Up to 100 m the choice is Cat6a with 10GBASE-T (IEEE 802.3an). From just over 100 m through 400 m the choice is OM4 multimode with 10GBASE-SR at 850 nm (IEEE 802.3ae). Beyond 400 m the choice is OS2 single-mode: 10GBASE-LR out to 10 km with a 6.2 dB budget, and 10GBASE-ER from there out to 40 km with an 11.1 dB budget. Past 40 km the plan warns that the span needs amplification or DWDM.

Loss is kilometres times 0.35 dB, plus 0.5 dB per connector, plus 0.1 dB per splice, plus a 3 dB margin. When the scenario omits connector and splice counts, the engine uses two connectors, no splices, and says so in the reason. A copper request past 100 m raises PhysicalDistanceViolationError rather than being clamped. A zero, negative, or non-numeric distance raises InvalidParameterError.

```
loss = 0.35 x km + 0.5 x connectors + 0.1 x splices + 3.0 dB margin
```

A long dark-fibre span can be inside the catalogue reach and still warn. Forty kilometres at 0.35 dB/km is already 14 dB before connectors, splices, and the margin, which is more than the ER budget. The metro preset demonstrates the point: its 35 km link estimates 16.25 dB against 11.1 dB, so the plan is kept and a warning is reported. The WAN preset has four inter-city spans (650, 1,200, 4,800, and 250 km) that all exceed the 40 km ER catalogue reach. Those warnings are intentional; the logical design is still valid, but the physical layer would use provider wavelengths or a DWDM line system.

The media table in the export records the link, distance, medium, transceiver, estimated loss, budget, and severity. The dashboard shows the same table, so a reviewer can challenge a choice without reading the source.

6. VLSM

IPv4 addresses are 32-bit integers. Requests are sorted by host count, largest first, with the original order kept for ties. A LAN block needs hosts + 2 addresses. The prefix length is 32 minus the smallest host-bit count that covers that need. The block is aligned with (address + size - 1) AND NOT (size - 1). If that alignment would pass 255.255.255.255, allocation raises AddressSpaceExhaustedError instead of wrapping to 0.0.0.0.

Each block records the network, prefix, mask, wildcard, broadcast, first and last usable address, gateway, usable count, and utilization. The gateway is the first usable address. The wildcard is the inverse of the mask, not the mask itself. A /31 follows RFC 3021 and has no broadcast. A /32 is a single host. Point-to-point helper modes allocate /30 or /31. After the last block, the plan checks independently that no two ranges overlap and reports wasted addresses inside the root.

The trace below is produced by the standalone allocator for a /24 with four requests. It shows the descending sort and the alignment of each block to its own size.

```
Allocation trace for 192.168.10.0/24 with hosts 100,50,20,10

request  100 hosts -> /25 (128 addresses) -> 192.168.10.0/25
request   50 hosts -> /26 ( 64 addresses) -> 192.168.10.128/26
request   20 hosts -> /27 ( 32 addresses) -> 192.168.10.192/27
request   10 hosts -> /28 ( 16 addresses) -> 192.168.10.224/28

usable addresses: 126 + 62 + 30 + 14 = 232
allocated block space: 128 + 64 + 32 + 16 = 240 (93.8% packing)
overlap check: PASS (four disjoint ranges, none crossing .255)
```

A fixed-length plan at /25 would fit only two of the four requests, and an unaligned allocator would leave the third request at .160 or push the fourth block past the end of the /24. The engine reports the gap after .240 as unused rather than silently reusing it.

Preset	Root	What is allocated
Campus	10.10.0.0/16	Data, Wi-Fi, voice, management, and servers. Five VLANs.
Metro	10.20.0.0/16	HQ users, trading, two remote user LANs, colo, and two /30 links.
WAN	10.64.0.0/14	Four city IoT LANs, a cloud VPC, and four /30 links.

Table 4. Preset roots and allocated blocks.

7. Routing and topology

Routing follows the inputs. A cloud boundary, or more than one autonomous system, selects multi-area OSPF plus eBGP (RFC 2328 and RFC 4271). eBGP requires local_asn and peer_asn; the engine does not invent them. Otherwise, more than one site with a link longer than 400 m, or 50 or more subnets, selects multi-area OSPF with a summary per site. The summary is the shortest prefix that covers that site and no other site. Every remaining case is OSPF Area 0 only.

Condition	Selected protocol	Reason
One AS, one site, <50 subnets	OSPF Area 0	No inter-area boundary exists; one area keeps the design and the lab simple.
Multiple sites with a link >400 m, or >=50 subnets	Multi-area OSPF	Each site becomes an area; area 0 carries inter-site traffic and each ABR summarises its own site.
Cloud boundary or more than one AS	Multi-area OSPF + eBGP	The provider domain is isolated at an AS boundary; internal prefixes are advertised selectively.

Table 5. Routing decision table.

Area numbers are assigned deterministically: the HQ or core site is area 0, and remote sites receive 1, 2, 3 in input order. When a site has a single /25, the summary is that exact /25; the engine records exact=True so the config generator can decide whether an area range statement adds anything. Ranges are emitted only on the ABR that owns the site LAN, never on the HQ router that has no interface in that area.

BGP is only enabled when the scenario crosses an AS boundary. The local AS is 65010 and the peer is 64512 in the WAN preset. The cloud VPC prefix is advertised by BGP and is explicitly skipped by the OSPF network list; this is the fix for the first audited fault.

RSTP (IEEE 802.1w) is selected when there are at most 16 VLANs and fewer than 50 switches. Above either limit the decision is MSTP (IEEE 802.1s). HSRP is selected when the scenario is resilient; the open alternative, VRRP (RFC 5798), is named in the decision. Topology is three-tier when endpoints are at least 1,000 or buildings are at least 4. Otherwise it is a collapsed core. Access switch count is the endpoint count divided by 48 ports, rounded up. Oversubscription is (ports per switch x access speed) / (uplinks x uplink speed), so the campus ratio is (48 x 1) / (2 x 10) = 2.4:1.

Preset	Tier	Control plane	STP	FHRP	Address use	Over-sub
campus	Three-tier	OSPF Area 0	RSTP	HSRP	69.4%	2.4:1
metro	Collapsed core	Multi-area OSPF	RSTP	HSRP	69.7%	2.4:1
wan	Three-tier	Multi-area OSPF + eBGP	RSTP	HSRP	78.0%	2.4:1

Table 6. Selected design per preset.

The comparison builds the natural plan and one alternate tier and routing mode. Each option lists advantages, disadvantages, trade-offs, and violations. Cost is computed only when the scenario supplies a price for the access, distribution, and core switch, so all three presets state that cost was not scored.

8. Dashboard result for the campus preset

The web page loads a preset or a custom distance, endpoint count, building count, and resiliency flag. Tabs show the overview, the decision log, the address plan, a tier diagram, and the per-device configuration. The campus run below selected three-tier and single-area OSPF, reported 2.4:1 oversubscription, and reported 69.4% address utilization. The alternate collapsed-core plan is shown beside it and is marked as a violation of the tier rule for 1,600 endpoints.

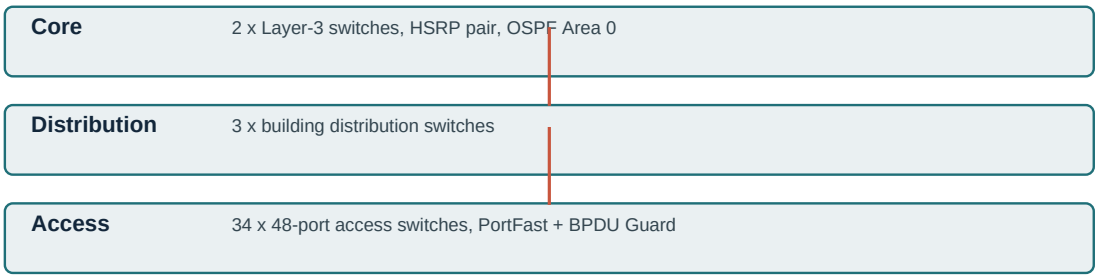


Figure 2. Campus preset: selected three-tier layout

Item	Selected	Alternate
Topology	Three-tier	Collapsed core
Routing	OSPF Area 0	Multi-area OSPF
Switches	39	36
Violations	0	1
Address utilization	69.4%	-

Table 7. Campus selected and alternate plans.

Tab	What it proves
Overview	Selected and alternate designs side by side, with the media table.
Decision log	Every choice carries a standard and a severity.
Address plan	CIDR, mask, wildcard, broadcast, gateway, usable hosts, and use.
Tier diagram	Core, distribution, and access counts from the topology plan.
Configurations	The exact IOS text written to the export, per device.

Table 8. Dashboard tabs.

The dashboard is not a second implementation. A test opens every tab and asserts that the rendered page contains a value computed by the engine, so a template that invents a number fails the suite. The alternate design is built by the same engine with the opposite tier and routing mode; it is a counterfactual check rather than a second guess.

9. Cisco IOS

configs.py fills one Jinja2 template, templates/ios_device.j2, once per device. Campus cores are Layer-3 switches with SVIs and HSRP: the virtual address is the first usable host, the first core takes the next address at priority 110, and the second core takes the following address at priority 100. Distribution and access switches are Layer-2. Where a scenario has point-to-point subnets, the routers own the Layer-3 interfaces and the switches stay Layer-2.

- Trunks list the allowed VLAN ids in numeric order and set native VLAN 999, which is not in the allowed list.
- Access ports enable PortFast and BPDU Guard. Campus voice and Wi-Fi ports also request inline power where the scenario marks PoE.
- Spanning tree mode follows the switching decision: rapid-pvst for the presets, mst when the engine selects MSTP.
- OSPF network statements use wildcard masks. Area range and BGP network statements use the subnet mask.
- User SVIs are passive. WAN adds eBGP and an ISAKMP crypto map on the cloud-facing interface only. Metro adds an LLQ policy for the trading subnet.
- An extended ACL denies the Wi-Fi subnet from reaching the server subnet. VTY lines accept SSH only, from the management subnet.
- Each core uses its own SVI address as the OSPF router ID, so an HSRP pair never presents duplicate IDs.

The files are written to exports/campus.txt, exports/metro.txt, and exports/wan.txt. The enable secret and the IPsec pre-shared key are lab strings for Packet Tracer. They are documented in the README and are not production credentials.

10. Tests

The suite lives in tests/ and contains 128 tests. It is organised by rule, so a failure names the engineering rule that broke rather than a source file. Media tests sit on the boundaries at 100 m, 101 m, 400 m, 401 m, 10 km, 40 km, and 50 km, and they expect an error when copper is requested past 100 m. VLSM tests cover exhaustion, an exact fit, a single host, /30, /31, and /32, descending sort, alignment that must not wrap, and wildcard masks at /0, /8, /24, /30, /31, and /32. A 1,000-run property test draws random subnet lists inside 10.0.0.0/16 and checks that nothing overlaps and nothing leaves the root. The ipaddress module is imported only in that test file.

Area	Representative assertions
Media	Distance boundaries; copper past 100 m raises; optical formula, LR/ER budgets, and the >40 km warning.
VLSM	Descending order, alignment, exact fit, exhaustion, /30, /31, /32, wildcards at /0 through /32, 1,000-run ipaddress cross-check.
Routing	Single-area OSPF, multi-area summaries, eBGP ASN requirement, 50-subnet threshold, cloud exclusion from OSPF.
Topology	Tier thresholds, forced collapsed-core violation, oversubscription formula.
Switching	RSTP/MSTP boundary, HSRP priorities, trunk VLAN ordering, native VLAN exclusion.
Configs	No allowed vlan all, no phantom WAN, unique router IDs, ACL and QoS presence, IPsec at the cloud boundary.
Integration	All three presets end to end, CLI exit codes, every dashboard route.

Table 9. Test coverage by engineering area.

Run the suite from the project folder. Exit codes are part of the CLI contract: 0 for success, 2 for invalid input or a boundary error, and 3 for address-space exhaustion.

```
python -m pytest -q
python -m pytest --cov=netarchitect.core --cov-report=term-missing
```

11. Packet Tracer lab

Each export is split by the hostname banner and is pasted into a matching device in Packet Tracer 8.x, from the hostname line through end, while the CLI is in configuration mode. Campus cores and distribution switches are 3560-24PS switches. Access switches are 2960-24TT switches. Metro and WAN routers are 2911 routers.

- Open the device, enter **enable**, then **configure terminal**.
- Paste the block from the hostname line through **end**.
- Save with **write memory**.
- The 3560-24PS model has GigabitEthernet0/1 and GigabitEthernet0/2 only. The generated third trunk, GigabitEthernet0/3, has to be applied to FastEthernet0/1 on that chassis and cabled there. Until that port is a trunk, the access link stays down and HSRP remains in Init because the VLAN interfaces have no forwarding port.
- On the 2911 used in the lab, HWIC-2T cards may enumerate as Serial0/2/0 and Serial0/3/0 rather than the generated names. Enter the address on the serial port the cable actually uses.
- The DCE end needs **clock rate** before line protocol comes up; the hub side already includes it.
- OSPF hellos need a short wait after line protocol comes up before **show ip ospf neighbor** lists the peers as FULL.
- The WAN export includes a cloud router, so the eBGP neighbour has a real peer in the generated file. Cable it to City A's cloud-facing interface.

Verification commands after pasting:

```
show vlan brief
show interfaces trunk
show standby brief
show ip ospf neighbor
show ip route ospf
show ip bgp summary
show crypto map
ping <gateway>
```

The manual lab itself is the remaining step: this report verifies the engine, the exports, and the tests, not a set of Packet Tracer screenshots. The README lists the same procedure and port caveats.

12. AI audit

ai_audit_log.md records four mistakes that appeared in generated output and were then fixed. The rubric asks for two; all four are included because each was reproduced from the generated IOS or a traceback. The log adds the prompt, the buggy code, the packet-flow consequence, the fix, and the regression test.

#	Fault	Forwarding effect	Fix
1	Cloud VPC prefix leaked into OSPF.	Cloud traffic matched an OSPF summary, bypassed eBGP, and avoided the IPsec crypto map.	Treat the cloud as area 0 and skip it in OSPF when BGP is used.
2	Area ranges applied on the HQ router, not the ABRs.	A widened prefix could be advertised by two areas, sending a packet back into its own area.	Emit each site's range only on the router that owns the LAN.
3	Both HSRP cores used the same OSPF router ID.	The cores never reached FULL, so inter-VLAN routing failed while HSRP still looked healthy.	Use each core's own SVI address as the router ID.
4	bgp read before assignment in routing.py.	analyze() crashed before writing any config; no packet flow existed.	Compute bgp before the network loop.

Table 10. AI-generated defects and their corrections.

The same log lists patterns checked on purpose even though they did not become faults: a wildcard copied from the subnet mask, a subnet aligned so carelessly that it wraps to 0.0.0.0, a trunk with no allowed-VLAN list, an OSPF network statement given a subnet mask, and an optical budget that forgets connector loss, splice loss, or the 3 dB margin.

The audit matters for academic integrity as much as for code quality. It shows that generated output was treated as a draft: each rule was checked against the specification, exercised through the CLI, inspected in the exported IOS, and then frozen by a test. Fault 3 is the most instructive because HSRP still showed Active/Standby while OSPF was silently broken; a superficial check would have passed it.

13. How to run the project

From the project folder, with Python 3.10 or newer, create the virtual environment, install the package, and run the commands below. The dashboard is served at http://127.0.0.1:8000 and the API documentation is at /docs.

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -e ".[dev]"
python -m netarchitect list-presets
python -m netarchitect explain campus
python -m netarchitect run --preset campus --out exports
python -m netarchitect run --preset metro  --out exports
python -m netarchitect run --preset wan    --out exports
python -m netarchitect serve --reload
```

Preset JSON files are netarchitect/presets/campus.json, metro.json, and wan.json. Generated IOS is under exports/. A custom scenario can be validated or run with --input, as in the branch-office example. The report itself is rebuilt with the optional report dependency:

```
python -m pip install -e ".[report]"
python tools/generate_report.py
```

14. Conclusion

NetArchitect keeps the planning rules in one place and makes both the CLI and the dashboard call that place. Media, VLSM, routing, and topology each return an explanation that names the input and the standard. The three presets produce different, checkable designs: a three-tier campus with one OSPF area, a collapsed metro with area summaries, and a WAN that adds eBGP at the cloud boundary. The test suite guards the distance boundaries, the address allocator, and the wildcard masks. The audit log keeps the four generation mistakes that were found and fixed, including the metro summary that would have looped a packet back into the wrong OSPF area.

The main lesson is that explainability is an engineering constraint, not a reporting feature. Once every number has to name its input and its standard, silent clamping and unsupported assumptions become visible. The 35 km metro span and the 4,800 km WAN span are good examples: the tool refuses to pretend that a catalogue reach is a working link.

References

[1] Aayush Kumar, NetArchitect source code, GitHub, https://github.com/Aayush3238/CN_Assignment, accessed 12 November 2026.

[2] IEEE 802.3an, Physical Layer and Management Parameters for 10 Gb/s Operation, Type 10GBASE-T.

[3] IEEE 802.3ae, 10 Gb/s Ethernet over fiber, including 10GBASE-SR, 10GBASE-LR, and 10GBASE-ER.

[4] IEEE 802.1Q, Bridges and Bridged Networks. IEEE 802.1w, Rapid Spanning Tree. IEEE 802.1s, Multiple Spanning Tree.

[5] RFC 1918, Address Allocation for Private Internets. RFC 4632, Classless Inter-domain Routing.

[6] RFC 2328, OSPF Version 2. RFC 4271, A Border Gateway Protocol 4. RFC 3021, Using 31-Bit Prefixes on IPv4 Point-to-Point Links.

[7] RFC 5798, Virtual Router Redundancy Protocol. Cisco IOS configurations in this project use HSRP.

[8] Cisco Systems, Catalyst 3560 and 2960 configuration guides; Cisco 2911 ISR configuration guide.

[9] CIC-307 NetArchitect assignment specification, 2026.

Appendix A. Generated configuration excerpt

The following block is taken verbatim from the campus core export. It shows the HSRP groups on the SVIs, the OSPF router ID and wildcard masks, and the management and guest ACLs. The full text is exports/campus.txt and the matching plan is exports/campus-plan.json.

```
! --- Data ---
interface Vlan10
 description Data
 ip address 10.10.0.2 255.255.252.0
 standby 10 ip 10.10.0.1
 standby 10 priority 110
 standby 10 preempt
 no shutdown
!
! --- Wi-Fi ---
interface Vlan30
 description Wi-Fi
 ip address 10.10.4.2 255.255.254.0
 standby 30 ip 10.10.4.1
 standby 30 priority 110
 standby 30 preempt
 no shutdown
!
! --- Voice ---
interface Vlan20
 description Voice
 ip address 10.10.6.2 255.255.254.0
 standby 20 ip 10.10.6.1
 standby 20 priority 110
 standby 20 preempt
 no shutdown
!
! --- Management ---
interface Vlan40
 description Management
 ip address 10.10.8.2 255.255.255.128
 standby 40 ip 10.10.8.1
 standby 40 priority 110
 standby 40 preempt
 no shutdown
!
! --- Servers ---
interface Vlan50
 description Servers
 ip address 10.10.8.130 255.255.255.128
 standby 50 ip 10.10.8.129
 standby 50 priority 110
 standby 50 preempt
 no shutdown
!
!
spanning-tree mode rapid-pvst
spanning-tree vlan 10,20,30,40,50 priority 4096
!
! --- OSPF ---
router ospf 1
 router-id 10.10.8.2
 network 10.10.0.0 0.0.3.255 area 0
 network 10.10.4.0 0.0.1.255 area 0
 network 10.10.6.0 0.0.1.255 area 0
 network 10.10.8.0 0.0.0.127 area 0
 network 10.10.8.128 0.0.0.127 area 0
 passive-interface Vlan10
 passive-interface Vlan30
 passive-interface Vlan20
 passive-interface Vlan40
 passive-interface Vlan50
!
ip access-list extended MGMT-ACCESS
 permit tcp 10.10.8.0 0.0.0.127 any eq 22
 deny ip any any
!
ip access-list extended GUEST-ISOLATION
 deny ip 10.10.4.0 0.0.1.255 10.10.8.128 0.0.0.127
 permit ip any any
!
interface Vlan30
 ip access-group GUEST-ISOLATION in
!
line con 0
 logging synchronous
line vty 0 4
 login local
 transport input ssh
 access-class MGMT-ACCESS in
!
end
```